

M.TECH DSAI · SEMESTER 3 · TECHNICAL PRESENTATION

LectureBank RAG Chatbot

Retrieval-Augmented Generation over the LectureBank NLP lecture corpus — with a dual cloud + local generation backend and a from-scratch automated evaluation suite.

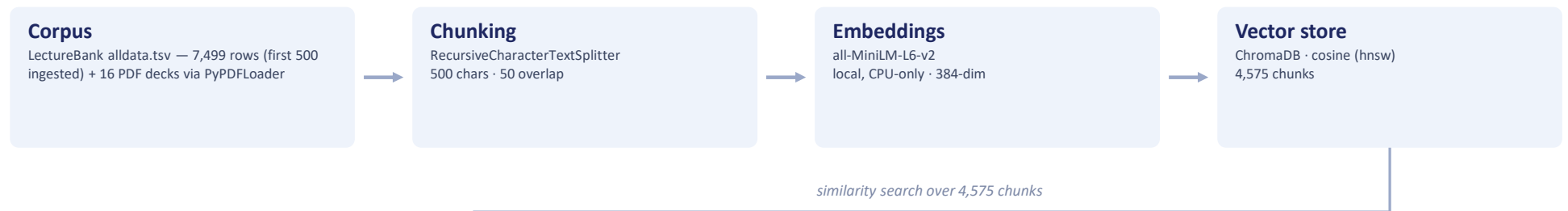


Yogesh A Gowda

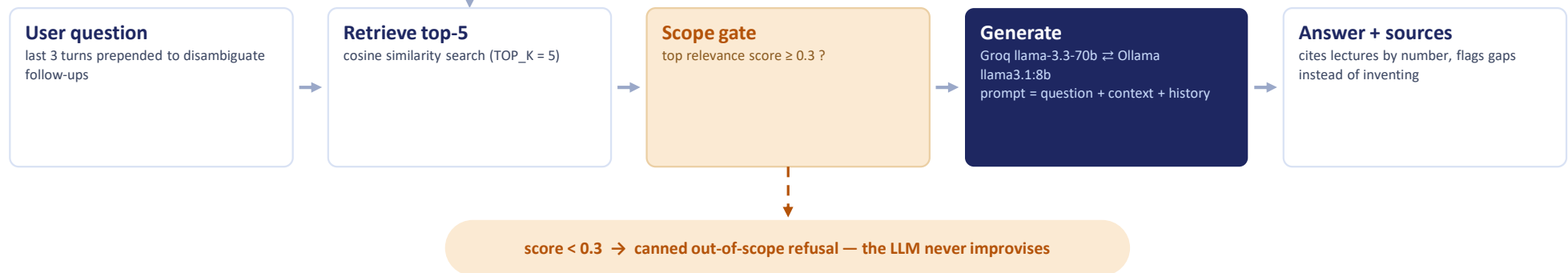
USN 24MDA098 · Indian Institute of Information Technology, Dharwad

System Architecture

INDEXING · OFFLINE



QUERY PATH · ONLINE



Serving — FastAPI /api/chat → static single-file vanilla JS/CSS/HTML UI with a Groq $\rightleftharpoons$ Ollama backend switcher (uvicorn, port 8000)

Data & Ingestion



500

TSV rows ingested

of 7,499 in alldata.tsv (MAX_ROWS cap)



16

PDF lecture decks

parsed with PyPDFLoader



4,575

chunks indexed

500-char chunks, 50-char overlap



384-d

local embeddings

all-MiniLM-L6-v2, CPU-only

How the index is built (free_rag_ingestion.py)

Synthetic documents — every TSV row becomes retrievable text: "Lecture / Instructor / University / Year / Topic / Slides URL".

Chunking — RecursiveCharacterTextSplitter with chunk_size = 500 and chunk_overlap = 50.

Embeddings — sentence-transformers/all-MiniLM-L6-v2: a local HuggingFace model, no API key, ~90 MB.

Vector store — ChromaDB persisted at db/chroma_db_free with hnsw:space = cosine.

Corpus: LectureBank (Yale LILY) — NLP lecture metadata plus 16 raw lecture slide decks.

Retrieval, Scope Control & Dual-Backend Generation



Retrieval & scope control (app.py)

Top-k retrieval — 5 chunks per query (TOP_K), cosine similarity over ChromaDB.

Conversational rewriting — the last 3 user turns are prepended to the question before retrieval, so follow-ups stay unambiguous.

Scope gate — if the top relevance score is below 0.3 (MIN_SCOPE_RELEVANCE), a canned out-of-scope reply is returned instead of letting the LLM improvise.

Conversation memory — in-memory deque per conversation_id, capped at 10 turns (MAX_TURNS).



Groq — cloud

llama-3.3-70b-versatile
temperature 0.1 · max_tokens 512 · free tier



Ollama — local

llama3.1:8b (default)
live model-availability check against localhost:11434

One prompt template for both backends: synthesize from the retrieved context, cite lectures by number, and flag gaps rather than invent content.



Serving — FastAPI + Pydantic: /api/chat · /api/status · /api/conversation/reset (uvicorn, port 8000)

UI — single self-contained static/index.html: vanilla HTML/CSS/JS, dark theme, Groq ⇌ Ollama switcher, served via StaticFiles.

Built From Scratch: the Automated Evaluation Suite

The original project shipped with no automated evaluation — answers were reviewed by eye. **A full eval/ package was added so quality is measured with numbers.**

Golden set

18 in-scope questions grounded in actually-indexed records (13 TSV rows + 5 named PDFs) + 5 out-of-scope, each with a known-correct source matcher.

`eval/golden_set.py`

Same code path as prod

A thin wrapper reuses `app.py`'s real `build_prompt / detect_scope` logic, so the eval exercises exactly what the live app runs — it cannot drift.

`eval/rag_pipeline.py`

Deterministic metrics

Hit rate, MRR, precision@k, and a keyword-coverage context-recall proxy — no LLM involved, fully repeatable.

`eval/metrics.py`

LLM-as-judge

Faithfulness, answer relevancy, context precision — judged by `openai/gpt-oss-20b`, deliberately a different model from the generator to reduce self-grading bias. A ragas stand-in (no C++ toolchain here).

`eval/llm_judge.py`

Runners

Retrieval-only eval, full generation eval, a Groq-vs-Ollama latency bench, and `run_all` — which orchestrates everything into a summary.

`eval/run_*.py`

Full per-question output

Every generated answer, judge verdict and unsupported-claim list is saved automatically on each run.

`eval/results/*.json`

Production logic untouched — `app.py` is only called, never modified.

What Each Metric Means



Retrieval *(deterministic)*

Hit Rate@5 · share of questions where a known-correct chunk appears in the top 5 — is the right lecture even findable?

MRR · mean of $1/\text{rank}$ of the first correct chunk — how high does the right content rank, on average?

Precision@5 · average share of the top 5 that is actually relevant — how much noise rides along with a hit?

Scope-gate accuracy · in-scope and out-of-scope questions correctly accepted / rejected by the 0.3 relevance cutoff.



Generation quality & speed

Faithfulness *(LLM-judged)* · the answer is split into atomic claims; each is checked against the retrieved context — is anything invented?

Answer relevancy *(LLM-judged)* · 0–1 score for how directly and completely the answer addresses the question.

Context precision *(LLM-judged)* · each retrieved chunk is marked relevant / irrelevant — of what was retrieved, how much was useful?

Context recall *(keyword proxy)* · share of expected topical keywords found anywhere in the retrieved context — needs no gold answers.

Latency *(measured)* · wall-clock retrieval and generation time — mean / median / p95.



Judge ≠ generator: scoring is done by openai/gpt-oss-20b — deliberately a different model from the llama-3.3-70b generator — to reduce self-grading bias.

Results — Retrieval Quality

94.4%

Hit Rate@5

a correct chunk lands in the top 5

0.787

MRR

first correct chunk ranks high

35.6%

Precision@5

relevant share of the top 5

95.7%

Scope-gate accuracy

22 / 23 questions correct

20.9 ms

Mean retrieval latency

ChromaDB top-5 search



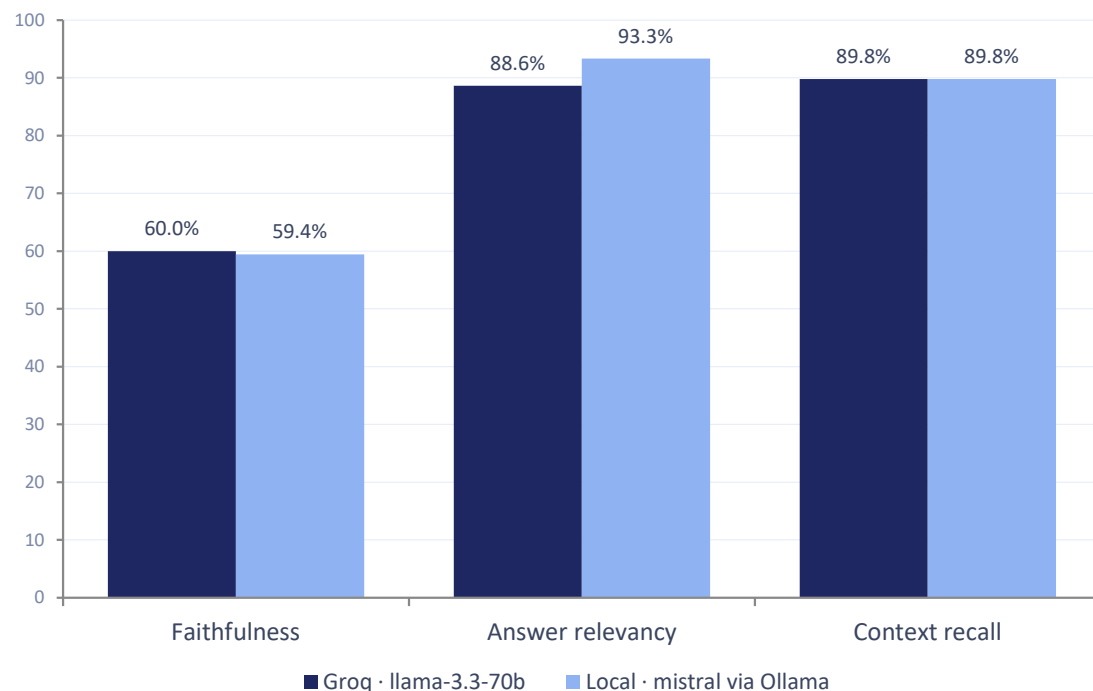
The right lecture is almost always findable — but it drags noise along. At 35.6% precision, roughly 3 of every 5 retrieved chunks aren't actually relevant. The natural next improvement: a re-ranking stage, or a smaller k.



The scope gate isn't airtight: one out-of-scope question — a cookie recipe — was incorrectly accepted as in-scope. A known limitation, not a solved problem.

Retrieval-only run over the golden set: 18 in-scope + 5 out-of-scope questions — no LLM calls involved.

Results — Generation Quality (LLM-Judged)



18-question golden set · judge: openai/gpt-oss-20b · context recall is a keyword proxy

Faithfulness is the weak spot

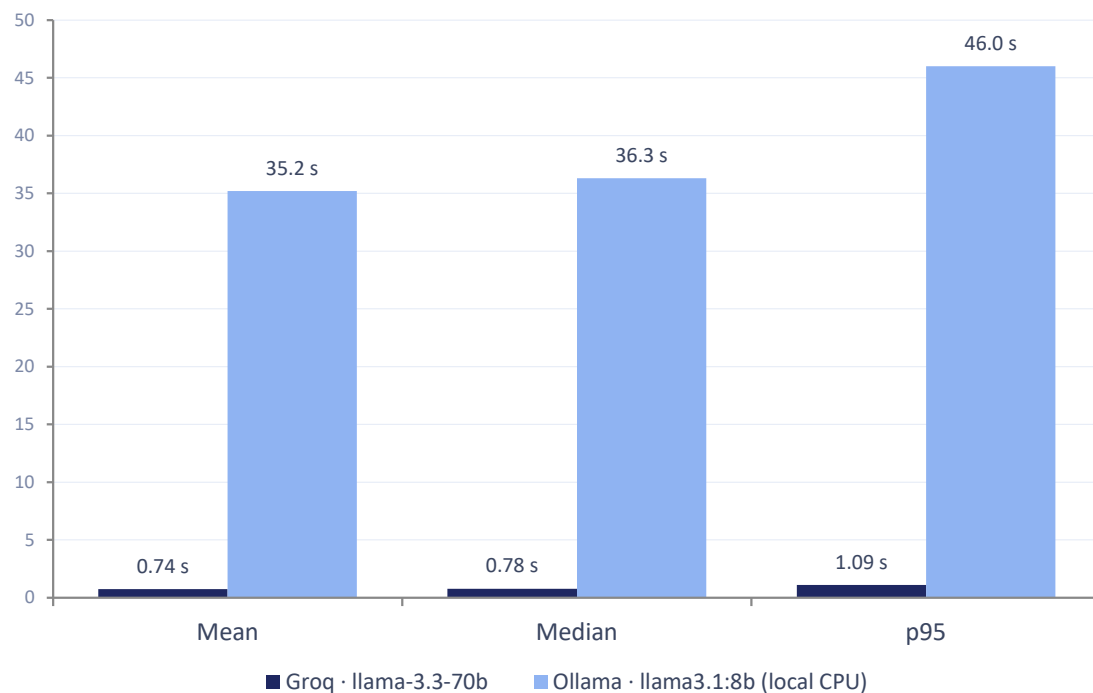
60.0% (Groq) vs 59.4% (local) — swapping the local model doesn't fix grounding; it's a different model failing in similar proportion, at ~57× the latency (884 ms → ~50.4 s mean).

Context precision 40.0% on the Groq run (not computed for the local run).

What the judge caught that a skim would miss:

- Ungrounded lecture-to-instructor attributions (faithfulness 0.0).
- Confident quotes & steps for Naive Bayes that were not in the retrieved context — fluent, plausible, wrong (0.0).
- An RNN answer degenerating into duplicated raw PDF-extraction fragments — a real chunking defect (0.25 faithfulness, 0.0 relevancy).

Latency — Groq Cloud vs Local Ollama



End-to-end generation time, 8 identical prompts per backend.

≈ 47.5×

faster mean response on Groq cloud than local CPU inference on this machine.

Groq · mean 740 ms · p95 1.09 s

Ollama · mean 35.2 s · p95 46.0 s

The build-vs-buy tradeoff: cloud speed against an external API dependency — with local inference as the fully-offline fallback.

Note: this benchmark ran llama3.1:8b locally; the generation-quality comparison (previous slide) used mistral:latest via a separate script.

Key Findings & Limitations



Retrieval is strong but noisy

94.4% hit rate, yet ~3 of 5 retrieved chunks are irrelevant (35.6% precision). Next step: a re-ranking stage or smaller k.



Faithfulness is the weakest score

60.0% — automated judging caught fluent, plausible, wrong answers that a human skim would have passed.



The scope gate isn't airtight

95.7% accuracy, but one cookie-recipe question was accepted as in-scope. A known limitation, not a solved problem.



Cloud vs local is a genuine tradeoff

~47.5× faster on Groq at the cost of an API dependency — and swapping the local model buys no faithfulness gain.

Say this with the numbers

- “On an 18-question golden set” — demo-sized, not a statistically rigorous sample.
- Context recall is a keyword-coverage proxy, not ragas’ LLM-based context_recall — no gold reference answers exist for this dataset.
- Faithfulness, relevancy and context precision are LLM-judged — label them that way, not as “measured”.

>_ Reproduce everything: `.venv\Scripts\python.exe -m eval.run_all` → full per-question detail in `eval/results/*.json`